NM i AI 2026 — Technical Cheat Sheet: Iteration 2

Fills gaps from Iteration 1. Read both documents together. March 19–22, 2026.

Quick Navigation

- Gap 1: Race Car — Real Implementation
- Gap 2: Healthcare RAG — Robust True/False Verification
- Gap 3: Tumor Segmentation — Pretrained & No-Training Approaches
- Gap 4: Grocery Bot — Advanced Multi-Agent Strategies
- Section A: Mathematical Scoring Analysis
- Section B: Hardware & Environment Optimization
- Section C: Competition Mindset & Decision Framework
- Quality Improvements: API, Debugging, LLM Dev, Norwegian

GAP 1: Race Car — Real Implementation

1.1 Competition API Patterns — How to Structure Your Agent Loop

Without a local simulator, you interact with the competition server via one of three patterns:

Pattern	How it works	Agent loop structure
HTTP polling	GET current state → POST action	while True: state=get(); action=decide(state); post(action)
WebSocket	Server pushes state frames; you respond	async for frame in ws: await ws.send(action)
Long-poll / SSE	Server holds connection open, streams events	for event in stream: post(event.id, action)

Assume WebSocket (most competition frameworks use it). Adapt the template from Iteration 1. If it is HTTP polling, wrap with the pattern below:

```
import requests, time, json

BASE_URL = "https://api.competition.no/racecar"
HEADERS = {"Authorization": "Bearer YOUR_TOKEN", "Content-Type": "applic

def get_state(session_id):
    r = requests.get(f"{BASE_URL}/state/{session_id}", headers=HEADERS, time
    r.raise_for_status()
    return r.json()

def post_action(session_id, action):
    r = requests.post(
        f"{BASE_URL}/action/{session_id}",
        headers=HEADERS,
        json=action,
        timeout=1.5
    )
    return r.json()

def run_http_agent(session_id):
    consecutive_errors = 0
    while True:
```

```
    try:
        state = get_state(session_id)
        if state.get("finished"):
            print(f"Race ended. Distance: {state.get('distance')}")
            break
        sensors = state["sensors"]  # list of 16 floats
        action = compute_action(sensors)
        result = post_action(session_id, action)
        consecutive_errors = 0
        time.sleep(0.05)  # avoid hammering API; tune to tick rate
    except requests.exceptions.Timeout:
        consecutive_errors += 1
        print(f"Timeout #{consecutive_errors}")
        if consecutive_errors > 5:
            print("Too many timeouts — exiting")
            break
    except requests.exceptions.HTTPError as e:
        print(f"HTTP error: {e.response.status_code} {e.response.text}")
        time.sleep(1)
    except Exception as e:
        print(f"Unexpected: {e}")
        time.sleep(0.5)
```

1.2 Finite State Machine (FSM) — Full Implementation

Implementation time: ~20 minutes. Tuning time: 1-2 hours via live API. [1][2]

An FSM is the cleanest architecture for sensor-based driving when you cannot train RL. Three states: **CRUISE** (forward is clear), **CORNER** (forward is partially blocked, steer away), **EMERGENCY** (imminent collision, brake hard).

```
from enum import Enum, auto
import numpy as np
```

```python
class DriveState(Enum):
    CRUISE    = auto()  # forward clear, go fast
    CORNER    = auto()  # wall ahead, steer toward open side
    EMERGENCY = auto()  # imminent crash, brake + sharp turn

class FSMRaceCar:
    """
    16-sensor FSM race car agent.
    Sensor layout assumption: sensor=forward, then clockwise.
    Adjust SENSOR_FORWARD / SENSOR_LEFT / SENSOR_RIGHT indices to mat
    """
    # Sensor index groups (adjust based on competition spec)
    SENSOR_FORWARD = [0, 1, 15]      # front-facing sensors
    SENSOR_LEFT    = [3, 4, 5]       # left-side sensors
    SENSOR_RIGHT   = [10, 11, 12]    # right-side sensors
    SENSOR_BACK    = [7, 8, 9]       # rear sensors (optional)

    # Thresholds (tune via binary search on live API)
    EMERGENCY_DIST = 8.0            # below this → EMERGENCY
    CORNER_DIST    = 25.0           # below this → CORNER
    CRUISE_DIST    = 40.0           # above this → CRUISE at full throttle

    # Action ranges (adjust if API uses different scale)
    MAX_THROTTLE   = 1.0
    CRUISE_THROTTLE = 0.9
    CORNER_THROTTLE = 0.4
    EMERGENCY_THROTTLE = -0.3        # brake / reverse

    def __init__(self):
        self.state = DriveState.CRUISE
        self.prev_steering = 0.0
        self.stuck_counter = 0
        self.last_action = {"steering": 0.0, "throttle": 0.5}

    def _group_min(self, sensors, indices):
        """Minimum value over a group of sensor indices."""
        return min(sensors[i] for i in indices if i < len(sensors))
```

```python
    def _group_mean(self, sensors, indices):
        vals = [sensors[i] for i in indices if i < len(sensors)]
        return sum(vals) / len(vals) if vals else 0.0

    def _update_state(self, sensors):
        fwd = self._group_min(sensors, self.SENSOR_FORWARD)
        if fwd < self.EMERGENCY_DIST:
            self.state = DriveState.EMERGENCY
        elif fwd < self.CORNER_DIST:
            self.state = DriveState.CORNER
        else:
            self.state = DriveState.CRUISE

    def compute_action(self, sensors):
        """
        sensors: list of 16 floats (raw distance readings)
        Returns: {"steering": float in [-1,1], "throttle": float in [-1,1]}
        """
        if len(sensors) != 16:
            return self.last_action  # safe fallback

        sensors = [float(s) for s in sensors]
        self._update_state(sensors)

        fwd   = self._group_min(sensors, self.SENSOR_FORWARD)
        left  = self._group_mean(sensors, self.SENSOR_LEFT)
        right = self._group_mean(sensors, self.SENSOR_RIGHT)

        if self.state == DriveState.CRUISE:
            throttle = self.CRUISE_THROTTLE
            # Smooth center-lane keeping
            balance = (left - right) / (left + right + 1e-6)
            steering = -0.3 * balance  # small correction
            # Speed boost when truly clear
            if fwd > self.CRUISE_DIST * 1.5:
                throttle = self.MAX_THROTTLE

        elif self.state == DriveState.CORNER:
```

```
            throttle = self.CORNER_THROTTLE
            # Steer toward the side with more clearance
            if left > right:
                steering = -0.7  # turn left
            else:
                steering = 0.7   # turn right
            # Proportional correction: harder turn when closer
            urgency = 1.0 - (fwd / self.CORNER_DIST)
            steering *= (0.5 + 0.5 * urgency)

        else:  # EMERGENCY
            throttle = self.EMERGENCY_THROTTLE
            # Find the most open quadrant
            all_sensors = list(enumerate(sensors))
            best_idx, best_val = max(all_sensors, key=lambda x: x[1])
            n = len(sensors)
            # Map index to [-1, 1] steering
            normalized = best_idx / n  # 0=forward, 0.5=backward
            if normalized <= 0.5:
                steering = (normalized - 0.125) * 4  # forward-biased
            else:
                steering = ((1.0 - normalized) - 0.125) * 4
            steering = float(np.clip(steering, -1.0, 1.0))
            self.stuck_counter += 1

        # Smooth steering to avoid jerky movements
        alpha = 0.6  # smoothing factor (lower = smoother, higher = more respons
        steering = alpha * steering + (1 - alpha) * self.prev_steering
        steering = float(np.clip(steering, -1.0, 1.0))
        throttle = float(np.clip(throttle, -1.0, 1.0))

        self.prev_steering = steering
        self.last_action = {"steering": steering, "throttle": throttle}
        return self.last_action

# Usage
agent = FSMRaceCar()
```

```
def handle_race_message(state):
    sensors = state["sensors"]
    return agent.compute_action(sensors)
```

1.3 Wall-Following as an Alternative Strategy

Wall-following guarantees the car never leaves a corridor — useful if the track has walls on both sides. The right-hand rule: always keep the right wall at a target distance.[2][1]

```
class WallFollower:
    """
    Right-hand wall following using PID control.
    Keeps right side at TARGET_DIST from the wall.
    Falls back to emergency steering when front is blocked.
    """
    TARGET_DIST    = 20.0   # desired distance from right wall (tune this)
    FRONT_BRAKE    = 15.0   # brake distance
    KP             = 0.04   # proportional gain
    KD             = 0.002  # derivative gain
    BASE_THROTTLE  = 0.7

    def __init__(self):
        self.prev_error = 0.0

    def compute_action(self, sensors):
        sensors = [float(s) for s in sensors]
        # Right-side sensors (adjust indices to your sensor layout)
        right_dist  = min(sensors[10], sensors[11])
        front_dist  = min(sensors, sensors[1], sensors[15])
        left_dist   = min(sensors[4], sensors[5])

        # Wall-following PID
        error = self.TARGET_DIST - right_dist  # positive = too close, negative = to
        d_error = error - self.prev_error
```

```
        steering = self.KP * error + self.KD * d_error
        self.prev_error = error

        # Emergency: front blocked → turn left (away from right wall)
        if front_dist < self.FRONT_BRAKE:
            steering = -0.9  # hard left
            throttle = 0.1
        else:
            throttle = self.BASE_THROTTLE * min(1.0, front_dist / 40.0)

        return {
            "steering": float(np.clip(steering, -1.0, 1.0)),
            "throttle": float(np.clip(throttle, 0.0, 1.0))
        }
```

When to use wall-following vs. FSM:

- **Wall-following**: Track has consistent walls; you want predictable behavior; first 15 minutes
- **FSM**: Track has open areas, intersections, or varying width; you want higher speed

---

1.4 Tuning Without a Local Simulator

Since the competition API may be the only way to evaluate, use **binary search on key thresholds**:

```
import json, time, requests

def evaluate_params(params, session_factory, n_runs=3):
    """
    Run the agent n_runs times with given params.
    Returns average distance.
    """
    distances = []
    agent = FSMRaceCar()
    # Override thresholds
```

```
    agent.EMERGENCY_DIST  = params["emergency_dist"]
    agent.CORNER_DIST     = params["corner_dist"]
    agent.CRUISE_THROTTLE = params["cruise_throttle"]

    for _ in range(n_runs):
        session_id = session_factory()  # creates new race session
        dist = run_race(agent, session_id)  # returns total distance
        distances.append(dist)
        time.sleep(2)  # cooldown between runs

    return sum(distances) / len(distances)

def binary_search_param(param_name, lo, hi, fixed_params, session_facto
    """Binary search a single continuous parameter."""
    results = []
    for _ in range(iters):
        mid = (lo + hi) / 2
        params = {**fixed_params, param_name: mid}
        score = evaluate_params(params, session_factory)
        results.append((mid, score))
        print(f"  {param_name}={mid:.2f} → score={score:.1f}")
        # Try both halves; keep the better one
        lo_score = evaluate_params({**fixed_params, param_name: lo}, session_
        hi_score = evaluate_params({**fixed_params, param_name: hi}, session_
        if lo_score > hi_score:
            hi = mid
        else:
            lo = mid
    best = max(results, key=lambda x: x[1])
    print(f"Best {param_name}: {best:.2f} (score {best[1]:.1f})")
    return best

# Tuning log: always log every run
def log_run(params, distance, filepath="tuning_log.jsonl"):
    with open(filepath, "a") as f:
        f.write(json.dumps({"params": params, "distance": distance, "ts": time.tim
```

Parameter tuning order:

1. EMERGENCY_DIST first (most impactful — prevents crashes)
2. CORNER_DIST second (balances speed vs. cornering)
3. CRUISE_THROTTLE last (tune once crash rate is low)

GAP 2: Healthcare RAG — Robust True/False Verification

2.1 NLI in Depth: From Scores to True/False[3][4][5]

NLI models output three logits: **contradiction** (index 0), **entailment** (index 1), **neutral** (index 2).

For fact-checking: entailment → TRUE, contradiction → FALSE, neutral → uncertain (default FALSE).

```
from sentence_transformers import CrossEncoder
import numpy as np
import torch

# Load once at startup
# cross-encoder/nli-deberta-v3-small: 87.5% MNLI, ~50ms/pair on CPU
# cross-encoder/nli-deberta-v3-base:  90.0% MNLI, ~200ms/pair on CPU (more a
# For competition speed: use -small; switch to -base if accuracy matters more

nli_model = CrossEncoder(
    'cross-encoder/nli-deberta-v3-small',
    max_length=512,
    device='cuda' if torch.cuda.is_available() else 'cpu'
)
# Label order: [contradiction, entailment, neutral]
NLI_LABELS = ['contradiction', 'entailment', 'neutral']

def nli_verify(statement: str, retrieved_chunks: list[str]) -> dict:
    """
    Returns dict with keys: label, entailment_score, contradiction_score,
```

```
    neutral_score, confidence, is_true
    """
    if not retrieved_chunks:
        return {"is_true": False, "confidence": 0.0, "label": "no_context"}

    # Concatenate top-3 chunks as premise (max ~400 tokens total)
    context = " ".join(retrieved_chunks[:3])
    if len(context) > 1500:
        context = context[:1500]

    # NLI pair: (premise=context, hypothesis=statement)
    scores = nli_model.predict([(context, statement)])
    # scores shape: (3,) — [contradiction, entailment, neutral]
    probs = torch.softmax(torch.tensor(scores), dim=0).numpy()

    contradiction_prob = float(probs)
    entailment_prob    = float(probs[1])
    neutral_prob       = float(probs[2])

    # Decision: entailment > contradiction → TRUE; else FALSE
    # Ignore neutral (treat as false for conservative medical fact-checking)
    is_true = entailment_prob > contradiction_prob

    # Confidence: margin between top two classes
    sorted_probs = sorted([contradiction_prob, entailment_prob, neutral_prob
    confidence = sorted_probs - sorted_probs[1]  # 0 = uncertain, 1 = very confid

    return {
        "is_true": is_true,
        "label": "entailment" if is_true else "contradiction",
        "entailment_score": entailment_prob,
        "contradiction_score": contradiction_prob,
        "neutral_score": neutral_prob,
        "confidence": confidence
    }
```

2.2 Combined Retrieval + NLI Score

High retrieval similarity means the retrieved chunk is topically relevant. Low NLI confidence means the chunk doesn't clearly support/contradict. Combine both:

```
def combined_verify(statement: str, chunks_with_scores: list[tuple]) -> dict:
    """
    chunks_with_scores: list of (chunk_text, retrieval_cosine_score)
    Returns unified decision with confidence.
    """
    if not chunks_with_scores:
        return {"is_true": False, "confidence": 0.0, "source": "no_context"}

    texts  = [c for c in chunks_with_scores]
    scores = [c[1] for c in chunks_with_scores]

    top_retrieval_score = float(scores)

    # Low retrieval confidence → the corpus doesn't cover this topic
    RETRIEVAL_THRESHOLD = 0.55
    if top_retrieval_score < RETRIEVAL_THRESHOLD:
        return {
            "is_true": False,
            "confidence": 0.0,
            "source": "low_retrieval",
            "note": f"Top chunk similarity only {top_retrieval_score:.2f} — insuffi
        }

    # Run NLI
    nli_result = nli_verify(statement, texts)

    # Weight NLI confidence by retrieval score
    # If retrieval is high (0.9) and NLI is confident → very reliable
    # If retrieval is borderline (0.6) and NLI is 50/50 → low combined confidence
    combined_confidence = nli_result["confidence"] * top_retrieval_score
```

```
    # Override rule: if retrieval is very high (>0.88) AND entailment score is high
    # strongly trust the TRUE verdict
    if top_retrieval_score > 0.88 and nli_result["entailment_score"] > 0.6:
        return {
            "is_true": True,
            "confidence": combined_confidence,
            "source": "high_retrieval_entailment"
        }

    return {
        "is_true": nli_result["is_true"],
        "confidence": combined_confidence,
        "source": "nli",
        "nli_detail": nli_result
    }
```

2.3 Handling Low-Confidence and Low-Retrieval Fallbacks

```
# Decision hierarchy:
# 1. If top_retrieval < 0.55 → default FALSE (not in corpus)
# 2. If NLI confidence < 0.15 → UNCERTAIN → default FALSE
# 3. If LLM available and confidence < 0.3 → escalate to LLM
# 4. Otherwise use NLI decision

def full_verify_pipeline(statement, index, chunks, model, nli_model, llm_cl
    query_emb = model.encode([statement], normalize_embeddings=True).ast
    scores, indices = index.search(query_emb, 5)
    retrieved = [(chunks[i], float(scores[j])) for j, i in enumerate(indices) if i >=

    result = combined_verify(statement, retrieved)

    # Escalate uncertain cases to LLM if available
    if result["confidence"] < 0.25 and llm_client is not None:
        llm_answer = verify_with_llm_cot(statement, [r for r in retrieved[:5]], llr
        result["is_true"] = llm_answer["is_true"]
```

```
        result["source"] = "llm_escalation"
        result["llm_reasoning"] = llm_answer.get("reasoning", "")

    return result
```

2.4 Chain-of-Thought Prompt for Medical Verification

```
def verify_with_llm_cot(statement: str, chunks: list[str], client) -> dict:
    """
    Chain-of-thought medical fact verification.
    Use gpt-4o-mini or similar for speed + cost efficiency.
    """
    context = "\n\n".join([f"Document {i+1}: {c}" for i, c in enumerate(chunks[:4

    system_prompt = """You are a precise medical fact-checker.
Your job is to determine if a medical statement is TRUE or FALSE based only
Follow this exact reasoning format:
1. RELEVANT EVIDENCE: Quote the most relevant sentence from the context.
2. ANALYSIS: Explain in one sentence whether it supports or contradicts the
3. VERDICT: TRUE or FALSE (one word only on this line)"""

    user_prompt = f"""Context documents:
{context}

Statement to verify: "{statement}"

Provide your step-by-step analysis:"""

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt}
        ],
        max_tokens=150,
```

```
        temperature=0
    )

    text = response.choices.message.content.strip()

    # Parse verdict from last meaningful line
    lines = [l.strip() for l in text.split("\n") if l.strip()]
    verdict_line = lines[-1].upper()
    is_true = "TRUE" in verdict_line and "FALSE" not in verdict_line

    # Extract reasoning for logging
    reasoning = text.split("VERDICT:").strip() if "VERDICT:" in text else text

    return {"is_true": is_true, "reasoning": reasoning, "raw": text}
```

2.5 Ensemble: When NLI and LLM Disagree

```
def ensemble_verify(statement, retrieved_chunks_with_scores, nli_model,
    """
    Two-model ensemble: NLI + LLM.
    Agreement → high confidence. Disagreement → use confidence to break ti
    """
    texts = [c for c in retrieved_chunks_with_scores]
    top_score = retrieved_chunks_with_scores[1] if retrieved_chunks_with_sc

    # Run both in parallel if possible
    nli_result = nli_verify(statement, texts)
    llm_result = verify_with_llm_cot(statement, texts[:4], llm_client)

    nli_true = nli_result["is_true"]
    llm_true = llm_result["is_true"]

    if nli_true == llm_true:
        # Agreement: high confidence
        return {
            "is_true": nli_true,
```

```
        "confidence": "high",
        "agreement": True
    }
else:
    # Disagreement: use NLI confidence score to break tie
    # NLI is more calibrated for entailment tasks; LLM better for complex rea
    # Conservative: default to FALSE on disagreement (reduces false positives
    nli_margin = abs(nli_result["entailment_score"] - nli_result["contradictic

    if nli_margin > 0.5 and top_score > 0.75:
        # NLI is very confident AND retrieval is good → trust NLI
        final = nli_true
        source = "nli_override"
    else:
        # Uncertain: default conservative FALSE
        final = False
        source = "conservative_fallback"

    return {
        "is_true": final,
        "confidence": "low",
        "agreement": False,
        "nli_vote": nli_true,
        "llm_vote": llm_true,
        "source": source
    }
```

2.6 Quick Calibration Against a Small Labeled Set

```
def calibrate(system, labeled_examples, threshold_range=(0.45, 0.90, 0.05)):
    """
    labeled_examples: list of {"statement": str, "label": bool}
    Finds the best retrieval threshold by sweeping.
    Takes ~2 minutes on 100 examples.
    """
```

```python
import numpy as np
from sklearn.metrics import f1_score, accuracy_score

thresholds = np.arange(*threshold_range)
best_thresh, best_f1 = thresholds, 0.0

# Get raw results for all examples first (avoid redundant API calls)
raw_results = []
for ex in labeled_examples:
    r = system.answer(ex["statement"])
    raw_results.append(r)

for thresh in thresholds:
    preds = [r["nli_detail"]["entailment_score"] > thresh
            if "nli_detail" in r else r["is_true"]
            for r in raw_results]
    labels = [ex["label"] for ex in labeled_examples]
    f1 = f1_score(labels, preds)
    acc = accuracy_score(labels, preds)
    print(f"Threshold {thresh:.2f} → F1={f1:.3f}, Acc={acc:.3f}")
    if f1 > best_f1:
        best_f1 = f1
        best_thresh = thresh

print(f"\nBest threshold: {best_thresh:.2f} (F1={best_f1:.3f})")
return best_thresh
```

GAP 3: Tumor Segmentation — Pretrained & No-Training Approaches

3.1 SUV Thresholding — Your Instant Baseline

Implementation time: 5 minutes. Dice score: ~0.4-0.6 for large tumors, ~0.2-0.3 for small.[6][7]

Before training anything, implement this threshold-based baseline. It often achieves 40-50% of what a trained model achieves and takes 5 minutes to code.

```
import numpy as np
from PIL import Image

def threshold_segment(image_path, method="percent_max", threshold=0.42
    """
    Simple SUV threshold segmentation.

    method options:
    - "fixed":       threshold = absolute SUV value (e.g., 2.5)
    - "percent_max": threshold = fraction of max intensity (e.g., 0.42 = 42% of S
    - "mean_plus_n": threshold = mean + n*std (e.g., n=2.0)

    Literature consensus: 42% of SUVmax gives best correlation with patholog
    for PET tumor delineation.
    """
    img = np.array(Image.open(image_path).convert('L'), dtype=np.float32)

    if method == "fixed":
        # Normalize if needed: assume img is scaled 0-255 representing 0-SUVmax
        mask = img > threshold
    elif method == "percent_max":
        suv_max = img.max()
        if suv_max == 0:
            return np.zeros_like(img, dtype=np.uint8)
        cutoff = threshold * suv_max  # e.g., 0.42 * SUVmax
        mask = img > cutoff
    elif method == "mean_plus_n":
```

```
        n = threshold  # treat threshold param as n
        cutoff = img.mean() + n * img.std()
        mask = img > cutoff
    else:
        raise ValueError(f"Unknown method: {method}")

    # Optional: morphological cleanup (remove noise)
    from scipy import ndimage
    mask = ndimage.binary_fill_holes(mask).astype(np.uint8)
    # Remove very small blobs (< 50 pixels)
    labeled, num = ndimage.label(mask)
    for i in range(1, num + 1):
        if (labeled == i).sum() < 50:
            mask[labeled == i] = 0

    return mask.astype(np.uint8)

# Quick evaluation
def quick_dice(pred_mask, true_mask):
    intersection = (pred_mask * true_mask).sum()
    return 2 * intersection / (pred_mask.sum() + true_mask.sum() + 1e-8)

# Try multiple thresholds on your validation set
for t in [0.30, 0.35, 0.40, 0.42, 0.45, 0.50]:
    mask = threshold_segment("test.png", method="percent_max", threshold=t
    # dice = quick_dice(mask, gt_mask)
    # print(f"Threshold {t:.2f} → Dice {dice:.3f}")
```

When to use this baseline:

- Before any training to establish a lower bound
- As a sanity check: if your U-Net scores lower than this, something is wrong
- As an ensemble component (average threshold + U-Net prediction)

3.2 SAM2 for Zero-Shot Segmentation

SAM2 (Segment Anything Model 2) can segment a MIP-PET tumor with no training by prompting it with a point at the brightest spot.[8][9][10]

```
# pip install sam2
# Download checkpoint: https://dl.fbaipublicfiles.com/segment_anything_2/092

import torch
import numpy as np
from PIL import Image
from sam2.build_sam import build_sam2
from sam2.sam2_image_predictor import SAM2ImagePredictor

def load_sam2_predictor(checkpoint_path, model_cfg, device=None):
    if device is None:
        device = "cuda" if torch.cuda.is_available() else "cpu"
    sam2 = build_sam2(model_cfg, checkpoint_path, device=device)
    predictor = SAM2ImagePredictor(sam2)
    return predictor

def segment_pet_with_sam2(image_path, predictor, n_points=3, threshold=
    """
    Zero-shot PET tumor segmentation.
    Strategy: click on the n brightest spots in the image.

    image_path: path to MIP-PET image (grayscale or RGB)
    predictor:  SAM2ImagePredictor instance (loaded once at startup)
    n_points:   number of brightest spots to use as prompts
    """
    # Load image
    img_np = np.array(Image.open(image_path).convert('L'), dtype=np.float32)
    # SAM2 expects RGB uint8
    img_rgb = np.stack([img_np] * 3, axis=-1)
    img_rgb = ((img_rgb - img_rgb.min()) / (img_rgb.max() - img_rgb.min() + 1e-
```

```
    # Find brightest spots as automatic prompts
    # Use 40% SUVmax threshold to find candidate bright regions
    thresh = img_np.max() * 0.40
    bright_mask = img_np > thresh
    ys, xs = np.where(bright_mask)

    if len(xs) == 0:
        # Fallback: click on absolute max
        flat_idx = np.argmax(img_np)
        y, x = np.unravel_index(flat_idx, img_np.shape)
        point_coords = np.array([[x, y]])
        point_labels = np.array([1])
    else:
        # Sample n_points from bright region (spaced out)
        if len(xs) > n_points:
            step = len(xs) // n_points
            sampled = range(0, len(xs), step)
            coords = [(xs[i], ys[i]) for i in sampled][:n_points]
        else:
            coords = list(zip(xs, ys))
        point_coords = np.array(coords)
        point_labels = np.ones(len(coords), dtype=int)  # all positive (foreground

    # Set image and predict
    predictor.set_image(img_rgb)

    with torch.inference_mode(), torch.autocast("cuda" if torch.cuda.is_availal
        masks, scores, logits = predictor.predict(
            point_coords=point_coords,
            point_labels=point_labels,
            multimask_output=True  # returns 3 masks; pick best
        )

    # Pick the highest-scoring mask
    best_mask_idx = int(np.argmax(scores))
    best_mask = masks[best_mask_idx].astype(np.uint8)

    return best_mask, float(scores[best_mask_idx])
```

```
# One-time model loading
PREDICTOR = load_sam2_predictor(
    checkpoint_path="./checkpoints/sam2.1_hiera_small.pt",
    model_cfg="configs/sam2.1/sam2.1_hiera_s.yaml"
)

# Per-image inference (fast: ~0.5s on GPU)
mask, confidence = segment_pet_with_sam2("pet_image.png", PREDICTOR)
```

SAM2 model size vs. speed tradeoff:

Model	Size	GPU Speed	CPU Speed	Recommended For
`sam2.1_hiera_tiny`	38.9M	~0.1s	~1s	Competition (fast)
`sam2.1_hiera_small`	46M	~0.2s	~2s	Best speed/quality
`sam2.1_hiera_large`	224M	~0.5s	~8s	Best accuracy

When to use SAM2 vs. training U-Net:

- **Use SAM2**: No training data, need results in < 1 hour
- **Use U-Net**: Have labeled data and 2+ hours to train; need consistent performance
- **Use both**: Ensemble SAM2 + U-Net threshold averaging often beats either alone

---

3.3 MONAI Pretrained Models — Load and Fine-Tune[11][12][13]

MONAI Model Zoo hosts pretrained bundles. The most relevant for PET-like data are the lesion/tumor segmentation models.

```python
from monai.bundle import ConfigParser, download
import torch

# Download a pretrained bundle from MONAI Model Zoo
# Available bundles: https://monai.io/model-zoo
# Closest to PET tumor: "lesion_mri_segmentation" or "brats_mri_segmentatio

# Option 1: Use MONAI Bundle API
download(name="brats_mri_segmentation", bundle_dir="./pretrained_bundl

parser = ConfigParser()
parser.read_config("./pretrained_bundles/brats_mri_segmentation/configs/inf
model = parser.get_parsed_content("network")
# Load pretrained weights
ckpt = torch.load("./pretrained_bundles/brats_mri_segmentation/models/mod
model.load_state_dict(ckpt)

# Option 2: Use timm + MONAI decoder (faster setup, ~30 min)
# Load ImageNet pretrained encoder (ResNet50) + MONAI U-Net decoder
import timm
from monai.networks.nets import UNet
from monai.networks.layers import Norm

def build_transfer_model(freeze_encoder=True):
    """
    Use ImageNet encoder weights for fast convergence.
    Fine-tune ONLY the decoder for 30 minutes.
    """
    # Build standard MONAI U-Net but swap encoder for pretrained backbone
    model = UNet(
        spatial_dims=2,
        in_channels=3,      # 3 channels to use ImageNet weights (duplicate grays
        out_channels=1,
        channels=(64, 128, 256, 512),
        strides=(2, 2, 2),
        num_res_units=2,
        norm=Norm.BATCH,
```

```python
    )
    return model

def prepare_for_transfer(grayscale_image):
    """Convert grayscale PET to 3-channel for ImageNet pretrained models."""
    if grayscale_image.shape == 1:
        return grayscale_image.repeat(3, 1, 1)  # (1,H,W) → (3,H,W)
    return grayscale_image

# Fine-tuning recipe: freeze early layers, train last 2 levels + decoder
def setup_finetuning(model, freeze_depth=2):
    """Freeze first N encoder stages, train rest."""
    # Get all named params
    all_params = list(model.named_parameters())
    frozen_count = 0
    for i, (name, param) in enumerate(all_params):
        # Freeze first N*20% of model
        if i < len(all_params) * (freeze_depth * 0.2):
            param.requires_grad = False
            frozen_count += 1
        else:
            param.requires_grad = True

    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
    total = sum(p.numel() for p in model.parameters())
    print(f"Frozen {frozen_count} params. Trainable: {trainable:,} / {total:,}")
    return model
```

3.4 MedSAM2 for Medical PET Segmentation[14][15][8]

MedSAM2 is SAM2 fine-tuned specifically on medical imaging data including PET, CT, and MRI — making it superior to vanilla SAM2 for this task.[14]

```
# Install
git clone https://github.com/bowang-lab/MedSAM2.git
cd MedSAM2
pip install -e .
# Download checkpoint from project page
```

```
# MedSAM2 usage (bounding-box prompted — most reliable for tumors)
# Give it a box around the brightest region as the prompt

def auto_bbox_prompt(img_np, percentile=95):
    """
    Automatically generate a bounding box prompt around high-uptake regio
    Works by thresholding at the Nth percentile of intensity.
    """
    cutoff = np.percentile(img_np, percentile)
    hot_mask = img_np > cutoff
    ys, xs = np.where(hot_mask)
    if len(xs) == 0:
        h, w = img_np.shape
        return [w//4, h//4, 3*w//4, 3*h//4]  # default center box
    return [int(xs.min()), int(ys.min()), int(xs.max()), int(ys.max())]  # [x1, y1, x

# With MedSAM2: use same SAM2ImagePredictor API but with medsam2 weig
# key difference: bboxes are more reliable prompts than points for medical use
def segment_with_medsam2_bbox(image_path, predictor):
    img_np = np.array(Image.open(image_path).convert('L'), dtype=np.float32)
    img_rgb = np.stack([img_np]*3, axis=-1)
    img_rgb = ((img_rgb / (img_rgb.max()+1e-8)) * 255).astype(np.uint8)

    bbox = auto_bbox_prompt(img_np, percentile=96)

    predictor.set_image(img_rgb)
    with torch.inference_mode():
        masks, scores, _ = predictor.predict(
            box=np.array([bbox]),  # shape (1, 4) — [x1, y1, x2, y2]
            multimask_output=False
```

```
        )

    return masks.astype(np.uint8), float(scores)
```

GAP 4: Grocery Bot — Advanced Multi-Agent Strategies

4.1 Token Passing (TP) Algorithm[16][17][18][19]

Token Passing is a decoupled MAPD algorithm designed specifically for warehouse pickup-and-delivery scenarios. It outperforms HCA* in practice because it naturally handles dynamic task assignment.[20][16]

Core idea: There is one shared "token" (a synchronized object). Only the agent holding the token can plan. Each free agent requests the token, finds the minimum-cost task, plans a collision-free path to it, registers that path in the token, then releases the token.

```
import heapq
from collections import defaultdict

class Token:
    """Shared synchronized state for Token Passing."""
    def __init__(self):
        self.agent_paths = {}         # agent_id -> list of (row, col, timestep)
        self.task_assignments = {}     # agent_id -> task_id
        self.unassigned_tasks = []     # list of task dicts

    def is_position_reserved(self, row, col, t):
        """Check if any agent has reserved (row, col) at time t."""
        for agent_id, path in self.agent_paths.items():
            for pr, pc, pt in path:
                if pr == row and pc == col and pt == t:
```

```
                return True
        return False

    def get_reserved_positions(self):
        """Returns set of (row, col, timestep) tuples reserved by all agents."""
        reserved = set()
        for path in self.agent_paths.values():
            for r, c, t in path:
                reserved.add((r, c, t))
        return reserved

def astar_with_token(grid, start, goal, token, agent_id, max_t=200):
    """
    A* in space-time, avoiding positions reserved in token.
    Returns list of (row, col) positions.
    """
    rows, cols = len(grid), len(grid)
    reserved = token.get_reserved_positions()

    def h(pos):
        return abs(pos-goal) + abs(pos[1]-goal[1])

    open_q = [(h(start), 0, start, 0, [start])]  # (f, g, pos, t, path)
    visited = set()

    while open_q:
        f, g, pos, t, path = heapq.heappop(open_q)
        state = (pos, t)
        if state in visited:
            continue
        visited.add(state)

        if pos == goal:
            # Return path as (row, col, timestep) for token registration
            return [(p, p[1], i) for i, p in enumerate(path)]

        if t >= max_t:
            continue
```

```
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1),(0,0)]:  # include wait
            nr, nc = pos+dr, pos[1]+dc
            npos = (nr, nc)
            if dr != 0 or dc != 0:
                if not (0 <= nr < rows and 0 <= nc < cols):
                    continue
                if not grid[nr][nc]:
                    continue
            else:
                npos = pos  # wait in place

            nt = t + 1
            if (nr, nc, nt) in reserved:
                continue

            new_path = path + [npos]
            new_g = g + 1
            heapq.heappush(open_q, (new_g + h(npos), new_g, npos, nt, new_patl

    return []  # No path found

def token_passing_assign(token, agents, grid):
    """
    One round of token passing: each free agent picks and plans its next task.
    Call this at the start of each game tick.
    """
    free_agents = [
        a for a in agents
        if a['id'] not in token.task_assignments and a['id'] not in [
            k for k, v in token.task_assignments.items() if v is not None
        ]
    ]

    for agent in free_agents:
        if not token.unassigned_tasks:
            break
```

```
        # Pick task with minimum Manhattan distance to agent
        agent_pos = (agent['row'], agent['col'])
        best_task = min(
            token.unassigned_tasks,
            key=lambda t: abs(t['pickup']-agent_pos) + abs(t['pickup'][1]-agent_pos
        )

        # Plan path: agent → pickup → delivery
        path_to_pickup = astar_with_token(grid, agent_pos, best_task['pickup'], t

        if not path_to_pickup:
            continue  # can't reach pickup; skip

        # Register path and assign task
        token.agent_paths[agent['id']] = path_to_pickup
        token.task_assignments[agent['id']] = best_task['id']
        token.unassigned_tasks.remove(best_task)

    return token
```

*Token Passing vs. HCA vs. CBS:**

Algorithm	Complexity	Agents	Dynamic Tasks	Competition Recommendation
HCA*	O(N · T · W)	Up to 20	Poor	Good baseline (Iteration 1)
Token Passing	O(N · T · W)	Up to 30	**Excellent**	**Use this for grocery bot**
CBS	O(2^N) worst case	Up to 10-15	Poor	Too slow for 20 agents

When NOT to use CBS: With 20 agents, CBS is exponential in the worst case. CBS is optimal but impractical for the Nightmare level. Use Token Passing with task swaps (TPTS) instead.[18][21][22]

4.2 Scoring Optimization: Mathematical Analysis

Scoring function: `S = items_delivered * 1 + completed_orders * 5`

For an order with k items: delivering all k items gives `k*1 + 5 = k+5` total. But delivering items from k different orders (without completing any) gives only `k*1` total.

Decision rule — when to fill capacity vs. deliver early:

Let's say a bot holds `m` items and needs `r` more items to complete an order. The detour cost to pick up those `r` items is `d` extra steps (time units).

Fill capacity and complete order **if:**

$$d \leq \frac{5}{\text{items_per_step}}$$

where `items_per_step ≈ 0.05` at medium difficulty (empirical estimate). So: fill if the detour is ≤ 100 steps.

In practice:

```
def should_complete_order_first(bot_pos, pending_order_items, drop_zon
    """
    Returns True if bot should pick up remaining order items before deliveri
    Uses +5 order bonus analysis.
    """
    if not pending_order_items:
        return False  # nothing to complete

    # Cost of going to pick up remaining items
    nearest_item = min(
        pending_order_items,
        key=lambda pos: abs(pos-bot_pos) + abs(pos[1]-bot_pos[1])
    )
    detour_cost = abs(nearest_item-bot_pos) + abs(nearest_item[1]-bot_pos[1])
```

```
    # Direct cost of going to drop zone
    drop_cost = abs(drop_zone_pos-bot_pos) + abs(drop_zone_pos[1]-bot_pos[1

    # Value gained from completing the order: +5 bonus - cost of detour
    # Each step ≈ 1 unit of time, during which you're not scoring anything else
    # With 300 rounds, each step costs roughly 1/300 of the game
    bonus_value = 5  # order completion bonus
    detour_penalty = detour_cost  # opportunity cost in steps

    # Complete order if bonus outweighs the roundtrip detour to items
    return bonus_value > (detour_cost - 0)  # simplification: complete if bonus

def max_theoretical_score(difficulty, rounds=300):
    """
    Rough theoretical maximum score per difficulty level.
    Assumptions: all bots always busy, 3-item orders, optimal routing.
    """
    config = {
        1: {"bots": 1,  "grid": (12,10),  "orders_visible": 2},
        2: {"bots": 5,  "grid": (18,14),  "orders_visible": 4},
        3: {"bots": 10, "grid": (22,16),  "orders_visible": 6},
        4: {"bots": 15, "grid": (26,18),  "orders_visible": 8},
        5: {"bots": 20, "grid": (30,18),  "orders_visible": 10},
    }
    c = config[difficulty]
    avg_path_length = (c["grid"] * c["grid"][1]) ** 0.5  # rough diagonal
    items_per_trip = 3
    trips_per_bot = rounds / (avg_path_length * 2)  # pickup + delivery
    items_per_bot = trips_per_bot * items_per_trip
    order_bonus_per_bot = trips_per_bot * 5  # assume each trip completes an o
    total = c["bots"] * (items_per_bot + order_bonus_per_bot)
    return int(total)
```

4.3 Proactive Pre-Fetching: Preview Order Strategy

If the API shows the next order before the current order is complete, proactively send idle bots to pre-position near the next order's items:

```
def plan_prefetch(active_order_items, next_order_items, idle_bots, token, g
    """
    Assign idle bots to move toward next order items while active order is bei
    Only pre-fetch if bot has no current assignment.
    """
    if not next_order_items or not idle_bots:
        return

    for bot, item_pos in zip(idle_bots, next_order_items):
        # Don't pre-fetch if item is far (>15 steps) — risk not being ready
        bot_pos = (bot['row'], bot['col'])
        dist = abs(item_pos-bot_pos) + abs(item_pos[1]-bot_pos[1])
        if dist <= 15:
            path = astar_with_token(grid, bot_pos, item_pos, token, bot['id'])
            if path:
                token.agent_paths[bot['id']] = path
                print(f"Bot {bot['id']} pre-fetching next order item at {item_pos}")
```

SECTION A: Mathematical Scoring Analysis

A.1 Grocery Bot — Theoretical Maximum

For a single bot on Easy (12×10 grid, 300 rounds):

- Average Manhattan distance per trip (pickup + delivery): ~10 steps
- Trips possible: 300/10 = 30 trips
- Items per trip: 3 → 90 items × 1 = 90 points

- Orders completed: ~30 trips / (order size ~3) = ~10 orders × 5 = 50 points
- **Single-bot theoretical max: ~140 points**

For 20 bots on Nightmare (30×18, 300 rounds):

- Average trip distance: ~20 steps
- Trips per bot: 300/20 = 15 trips → 15 × 3 = 45 items
- 20 bots × 45 items = 900 items + 20 bots × 15 × 5 = 1,500 order bonuses
- **20-bot theoretical max: ~2,400 points**

Key insight: The +5 order bonus is proportionally more valuable at low bot counts where item delivery rate is slow. At 20 bots, raw item throughput dominates.

A.2 Race Car — Optimal Throttle Function

Given sensor reading `f` (forward distance, normalized 0-1):

$$\text{throttle}(f) = \min\left(1.0,\ \frac{f}{f_{\text{brake}}}\right)$$

where f_{brake} is your CORNER_DIST threshold. This linear ramp ensures smooth deceleration. The steering response function:

$$\text{steering}(l, r) = K_p \cdot \frac{l - r}{l + r + \epsilon}$$

where l, r are mean left/right clearances and $\epsilon = 0.01$ prevents division by zero.

Distance maximization: distance ≈ throttle × time. With 60 seconds: `maximize integral(throttle(t) dt) subject to crash_probability(throttle, sensor_readings) < threshold`. This means: run high throttle only when all forward sensors are clear; early braking is always better than crashing.

A.3 RAG — Accuracy Requirements for Top Performance

In a binary classification task (true/false) with 115 topic classes:

For true/false: with a random 50/50 split baseline, you need:

- **Top 10%**: ~80% accuracy on true/false
- **Top 3%**: ~90%+ accuracy
- The NLI approach (deberta-v3-small) achieves ~87% on medical NLI tasks without fine-tuning

For topic classification: with 115 classes:

- Random baseline: 0.87%
- Zero-shot cosine similarity: ~45-55% accuracy (depends on topic label quality)
- Fine-tuned logistic regression on embeddings: ~70-80%
- **Top score likely comes from topic classification accuracy** — invest time here

---

SECTION B: Hardware & Environment Optimization

B.1 Maximize GPU Utilization During Overnight Training

```
# Check GPU utilization baseline
import subprocess
result = subprocess.run(["nvidia-smi", "--query-gpu=utilization.gpu,memory.
                         "--format=csv,noheader,nounits"], capture_output=True, text=T
print(result.stdout)

# Target: GPU utilization > 80%. If lower, increase batch size or num_workers.

from torch.utils.data import DataLoader
```

```
# Optimal DataLoader for GPU training
loader = DataLoader(
    dataset,
    batch_size=16,            # increase until OOM; start here
    num_workers=4,            # use 4 CPU workers for data loading
    pin_memory=True,          # faster CPU→GPU transfer
    prefetch_factor=2,        # prefetch 2 batches ahead
    persistent_workers=True,  # keep workers alive between epochs
)
```

B.2 Mixed Precision Training (2x Speedup) [23]

```
from torch.cuda.amp import GradScaler, autocast

scaler = GradScaler()

def train_step_amp(model, images, labels, optimizer, criterion):
    optimizer.zero_grad()
    with autocast():  # automatic mixed precision (float16 on GPU)
        outputs = model(images)
        loss = criterion(outputs, labels)
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    scaler.step(optimizer)
    scaler.update()
    return loss.item()
```

B.3 Inference Speedup: torch.compile, FP16, ONNX[24][25][23]

```
import torch
import torch.onnx
import onnxruntime as ort
import numpy as np

# ─── Method 1: torch.compile (easiest, 20-30% speedup) ───
model = torch.compile(model, mode="reduce-overhead")  # first call takes ~3

# ─── Method 2: FP16 inference (2x speedup on GPU) ───
model_fp16 = model.half()
with torch.no_grad():
    img_fp16 = img_tensor.half().to("cuda")
    output = model_fp16(img_fp16)

# ─── Method 3: ONNX export + OnnxRuntime (best CPU speedup) ───
def export_to_onnx(model, save_path, input_shape=(1, 1, 512, 512)):
    model.eval()
    dummy = torch.randn(*input_shape)
    torch.onnx.export(
        model, dummy, save_path,
        input_names=["input"],
        output_names=["output"],
        dynamic_axes={"input": {0: "batch_size"}, "output": {0: "batch_size"}},
        opset_version=17,
        do_constant_folding=True
    )
    print(f"Exported to {save_path}")

def onnx_infer(session, img_np):
    """img_np: float32 array (1, 1, H, W)"""
    outputs = session.run(None, {"input": img_np})
    logits = outputs
    probs = 1 / (1 + np.exp(-logits))  # sigmoid
    return (probs > 0.5).astype(np.uint8).squeeze()
```

```python
# Export once
export_to_onnx(model, "tumor_unet.onnx")

# Load for inference
providers = ["CUDAExecutionProvider", "CPUExecutionProvider"]
session = ort.InferenceSession("tumor_unet.onnx", providers=providers)
```

B.4 Memory Management — Avoiding OOM Errors

```python
# OOM diagnosis
torch.cuda.empty_cache()
print(f"Allocated: {torch.cuda.memory_allocated()/1e9:.2f} GB")
print(f"Reserved:  {torch.cuda.memory_reserved()/1e9:.2f} GB")

# Gradient accumulation for effectively larger batch sizes
ACCUMULATION_STEPS = 4  # effective batch = batch_size * 4

optimizer.zero_grad()
for i, batch in enumerate(loader):
    images, labels = batch["image"].to(device), batch["label"].to(device)
    with autocast():
        outputs = model(images)
        loss = criterion(outputs, labels) / ACCUMULATION_STEPS
    scaler.scale(loss).backward()

    if (i + 1) % ACCUMULATION_STEPS == 0:
        scaler.step(optimizer)
        scaler.update()
        optimizer.zero_grad()

# Apple Silicon MPS backend (for macOS users)
if torch.backends.mps.is_available():
    device = torch.device("mps")
    print("Using Apple Silicon MPS")
```

```
    # Note: not all MONAI ops support MPS; fallback to CPU for those
    try:
        model = model.to(device)
    except Exception:
        device = torch.device("cpu")
        model = model.to(device)
```

B.5 Process Priority for Overnight Training

```
# Linux: run training at lower IO priority to prevent system freeze
ionice -c 3 python train_tumor.py &

# Run at lower CPU priority
nice -n 10 python train_tumor.py

# Monitor GPU in real-time
watch -n 2 nvidia-smi

# Kill training if it OOMs without freezing the system
python train_tumor.py &
TRAIN_PID=$!
# Monitor memory and kill if >95% full
while true; do
  MEM=$(nvidia-smi --query-gpu=memory.used --format=csv,noheader,nouni
  MAX=$(nvidia-smi --query-gpu=memory.total --format=csv,noheader,nouni
  if [ $((MEM * 100 / MAX)) -gt 95 ]; then
    echo "GPU OOM risk — killing training"
    kill $TRAIN_PID
    break
  fi
  sleep 30
done
```

SECTION C: Competition Mindset & Decision Framework

C.1 Decision Tree: "My Score Is Not Improving"

```
Score not improving →
  ├─ Am I even submitting correctly?
  │    └─ Check: dummy submission with known output → score changes?
  │        ├─ No change: submission pipeline broken → fix first
  │        └─ Changes: logic problem, not infrastructure
  │
  ├─ Is my approach fundamentally wrong?
  │    └─ Check: does my output look qualitatively right?
  │       (Bot: are bots moving toward items? RAG: are retrieved chunks relev
  │        Tumor: does mask overlap visible bright spots?)
  │        ├─ No: wrong approach → pivot (see below)
  │        └─ Yes: parameter tuning / threshold problem → tune
  │
  ├─ Am I in a local optimum?
  │    └─ Symptom: small improvements for large changes in parameters
  │        ├─ Grocery Bot: try completely different assignment algorithm
  │        ├─ RAG: try different embedding model or chunking strategy
  │        ├─ Tumor: try SUV threshold ensemble vs. pure U-Net
  │        └─ Race Car: try wall-following instead of center-seeking
  │
  └─ Is the competition server behaving differently from expected?
     └─ Add extensive logging, compare your state parsing to competition sp
```

C.2 Decision Tree: "I Have 2 Hours Left"

```
2 hours left →
  ├─ Do I have a working submission for all 4 challenges?
  │    ├─ No → submit a dummy that doesn't crash for missing ones
```

```
│   └─ Yes → go to optimization
│
├─ Optimization priority:
│   1. Fix crashes/timeouts (anything that fails = 0 points)
│   2. Improve the challenge with largest gap to next rank
│   3. If gap is small on all: polish the highest-weight challenge
│
├─ Is there a quick win available?
│   ├─ RAG: tune NLI threshold on labeled set (30 min → +10% F1)
│   ├─ Tumor: try 42% SUVmax threshold ensemble (10 min → may beat br
│   ├─ Bot: add order completion prioritization (20 min → +5-15% score)
│   └─ Race Car: tune one FSM threshold (15 min via binary search)
│
└─ Do not: rewrite from scratch. Fix, tune, ensemble.
```

C.3 When to Abandon an Approach

Abandon when ALL three are true:

1. You've spent > 3 hours on it
2. Score hasn't improved in the last 1 hour despite changes
3. A fundamentally different approach exists and takes < 1 hour to implement

Pivot signals by challenge:

- **Grocery Bot**: If bots are constantly deadlocked → switch from HCA* to Token Passing
- **RAG**: If NLI scores are all ~0.5 → your chunking is wrong (chunks too large or wrong corpus)
- **Tumor**: If Dice < 0.15 after 2+ epochs → data loading bug (check mask values: should be {0, 1})
- **Race Car**: If car crashes within 5 seconds consistently → emergency threshold is too low

C.4 Energy Management Over 4 Days

- **Day 1 evening**: Stop by midnight. Fresh mind > late-night bug chasing.
- **Day 2**: Highest cognitive load day (architecture decisions). Make design choices before 3 PM.
- **Day 3**: Best for optimization loops (can run overnight). Start training at 6 PM.
- **Day 4**: Mechanical work only (no new architectures). Submit early, test submissions.

QUALITY IMPROVEMENTS

QI-1: Rapid Debugging Patterns

Mock WebSocket Server for Offline Testing

```python
import asyncio, json, websockets, random

# Minimal mock server to replay recorded game states
async def mock_game_server(websocket):
    """Replay a pre-recorded game state sequence."""
    states = [
        {"bots": [{"id": 0, "row": 2, "col": 3, "items": []}],
         "orders": [{"id": 1, "items": [{"row": 5, "col": 7}]}],
         "round": 1},
        # ... add more states
    ]
    for state in states:
        await websocket.send(json.dumps(state))
        response = await asyncio.wait_for(websocket.recv(), timeout=2.0)
        print(f"Bot responded: {response}")
        await asyncio.sleep(0.1)

async def main():
    async with websockets.serve(mock_game_server, "localhost", 8765):
        print("Mock server running on ws://localhost:8765")
```

```
        await asyncio.Future()  # run forever

asyncio.run(main())
```

RAG Evaluation Harness (20 lines)

```
def eval_rag(system, labeled: list[dict]) -> dict:
    """labeled: [{"statement": str, "label": bool, "topic_id": int}]"""
    tf_correct, topic_correct = 0, 0
    for ex in labeled:
        result = system.answer(ex["statement"])
        if result["is_true"] == ex["label"]:
            tf_correct += 1
        if result["topic_id"] == ex["topic_id"]:
            topic_correct += 1
    n = len(labeled)
    print(f"T/F accuracy:    {tf_correct}/{n} = {tf_correct/n:.1%}")
    print(f"Topic accuracy:  {topic_correct}/{n} = {topic_correct/n:.1%}")
    return {"tf_acc": tf_correct/n, "topic_acc": topic_correct/n}
```

Tumor Prediction Visualization (3 lines)

```
import matplotlib.pyplot as plt
import numpy as np

def vis_prediction(img_path, pred_mask, gt_mask=None):
    img = np.array(__import__('PIL').Image.open(img_path).convert('L'))
    fig, axes = plt.subplots(1, 3 if gt_mask is not None else 2, figsize=(12, 4))
    axes.imshow(img, cmap='hot'); axes.set_title('PET Image')
    axes[1].imshow(pred_mask, cmap='Reds', alpha=0.7); axes[1].set_title('Pred
    if gt_mask is not None:
        overlay = np.stack([img/img.max()]*3, axis=-1)
        overlay[:,:,0] += pred_mask.astype(float) * 0.5   # red = prediction
        overlay[:,:,2] += gt_mask.astype(float) * 0.5     # blue = ground truth
        axes[2].imshow(np.clip(overlay, 0, 1)); axes[2].set_title('Overlay (red=pre
    plt.tight_layout(); plt.savefig('prediction_vis.png', dpi=100); plt.show()
```

QI-2: Norwegian Language Considerations for RAG[26][27][28][29]

Norwegian Embedding Model Comparison

Model	Hugging Face ID	Size	Norwegian NLU (F1)	Medical Text	Use When
NB-BERT-base	NbAiLab/nb-bert-base	110M	**85.9**	General	Text is mostly Norwegian
NB-BERT-large	NbAiLab/nb-bert-large	340M	**87.0**	General	Need highest accuracy
NorBERT3-base	ltg/norbert3-base	123M	~85	General	Fast alternative to NB-BERT
NorDeClin-BERT	(not yet public)	110M	N/A	**Clinical**	Norwegian clinical notes
multilingual-e5-base	intfloat/multilingual-e5-base	270M	~82	General	Mixed NO/EN corpus
paraphrase-multilingual-MiniLM-L12	sentence-transformers/...	118M	~78	General	Speed priority

Recommendation for competition:

- If corpus is **English medical text** + statements in English: use all-MiniLM-L6-v2 (fastest)

- If corpus or statements contain **Norwegian**: use `paraphrase-multilingual-MiniLM-L12-v2` — it's fast, handles Norwegian, and avoids model-switching overhead
- Do NOT use `NB-BERT-base` directly for RAG — it's not fine-tuned for sentence embeddings; wrap it with `SentenceTransformer` or use as a cross-encoder only

Translate vs. Embed Directly

```
# Option 1: Embed Norwegian statements directly (faster, lower accuracy)
from sentence_transformers import SentenceTransformer
no_model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

# Option 2: Translate to English first, then embed (slower, higher accuracy on
from deep_translator import GoogleTranslator

def translate_if_norwegian(text: str, threshold=0.5) -> str:
    """
    Heuristic: if text contains Norwegian characters or common NO words, tra
    """
    norwegian_indicators = ['og', 'er', 'ikke', 'med', 'på', 'det', 'å', 'ø', 'æ']
    words = text.lower().split()
    no_ratio = sum(1 for w in words if w in norwegian_indicators) / max(len(
    if no_ratio > threshold or any(c in text for c in 'æøå'):
        try:
            return GoogleTranslator(source='no', target='en').translate(text)
        except Exception:
            return text  # fallback: use original
    return text

# Norwegian medical stopwords to remove before embedding
NO_MEDICAL_STOPWORDS = {
    'pasienten', 'pasient', 'legen', 'lege', 'behandling', 'sykehus',
    'klinisk', 'medisinsk', 'symptom', 'symptomer', 'diagnose'
}

def clean_norwegian_medical(text: str) -> str:
    words = text.split()
```

```
    cleaned = [w for w in words if w.lower() not in NO_MEDICAL_STOPWORD
    return ' '.join(cleaned)
```

QI-3: LLM-Assisted Development in Competition

CLAUDE.md Setup

Create a `CLAUDE.md` file in your repo root. This file is read by Claude Code / Cursor on startup and gives the AI context it needs to write accurate code for your project:

```
# NM i AI 2026 — Competition Repository

## Project Structure
- challenge1_bot/       → Grocery Bot WebSocket client
- challenge2_car/       → Race Car FSM agent
- challenge3_rag/       → Healthcare RAG pipeline
- challenge4_tumor/     → MIP-PET tumor segmentation

## Competition API
- Grocery Bot WS: ws://[URL]/bot (JSON state → JSON action)
- Race Car WS:    ws://[URL]/car (JSON {"sensors": [16 floats]} → {"steering": flo
- RAG API:        POST /classify {"statement": str} → {"is_true": bool, "topic_id": i
- Tumor API:      POST /segment (multipart image) → {"mask": base64_png}

## Key Constraints
- Grocery Bot: 2-second response timeout, max 3 items per bot
- Race Car: 1-minute race, 16 sensors (layout: sensor=forward, clockwise)
- RAG: <500ms per query, 115 topics
- Tumor: <10 seconds per image

## Environment
- Python 3.11
- PyTorch 2.3 with CUDA 12.1
- MONAI 1.3
- sentence-transformers 3.0
```

```
## Current Status
[Update this daily so the AI assistant always knows where you are]
- Bot: HCA* working, order batching TODO
- RAG: FAISS index built, NLI model loading TODO
- Tumor: U-Net training, epoch 5/50
- Car: FSM working, tuning needed
```

Effective LLM Prompts for Competition Code

"Debug this" pattern:

```
Here is my code that should [do X]:
[paste code]

When I run it I get:
[paste exact error + traceback]

The input data looks like: [paste example]
The expected output is: [describe]

Fix only the bug, don't refactor the code.
```

"Optimize for speed" pattern:

```
This function runs in [X ms] but needs to run in under [Y ms]:
[paste function]

It's called [N times/request] with inputs like: [example]
Profile output shows [hotspot].

Make it faster. Acceptable tradeoffs: [list what you can sacrifice].
Do NOT change the function signature.
```

"Adapt to my API" pattern:

```
Here is a working example using library X:
[example code]

My competition API works differently:
- Input format: [JSON schema or example]
- Output expected: [JSON schema or example]
- Authentication: [how headers look]

Rewrite ONLY the API interaction parts to match my format.
```

When LLM Suggestions Are Dangerous

Situation	Risk	How to verify
Hallucinated MONAI API	Model may invent non-existent transforms	Run `help(monai.transforms.X)` or check `dir(monai)`
Wrong library versions	Code that worked in 2023 may be deprecated	Check pip show [library] for version, then look at release notes
Invented model names	e.g., "monai.networks.nets.MedSAMPET" doesn't exist	Always `import` and `print(dir(module))` before trusting
Wrong tensor shapes	LLM may assume (B,C,H,W) when API needs (B,H,W,C)	Print tensor.shape at every step
Off-by-one in sensor indices	LLM may guess wrong sensor layout	Always test with sensor reading = [0,0,...,0,100] and verify behavior

---

Good luck at NM i AI 2026, Sander! The biggest delta from Iteration 1: for the Bot, switch from HCA to Token Passing as soon as you hit 10+ bots. For RAG, the NLI + retrieval score combination is significantly more robust than the cosine threshold. For tumors, try the 42% SUVmax baseline first — it may surprise you.*

References

1. Introduction to Wall Following — XRP 2023.0 documentation - A program to keep a certain distance from an object, let's implement this by having the XRP follow a...
2. Arduino Alvik Maze Navigation - From Wall Following to ... - Learn how to navigate an Arduino Alvik robot through a maze, starting with simple wall following and...
3. How to Use the Cross-Encoder for Natural Language Inference fxis.ai - How to Use the Cross-Encoder for Natural Language Inference
4. cross-encoder/nli-deberta-v3-base - We’re on a journey to advance and democratize artificial intelligence through open source and open s...
5. cross-encoder/nli-deberta-v3-small - We’re on a journey to advance and democratize artificial intelligence through open source and open s...
6. Intensity threshold based solid tumour segmentation method for Positron Emission Tomography (PET) images: A review - Accurate, robust and reproducible delineation of tumour in Positron Emission Tomography (PET) is ess...
7. FDG PET Metabolic Tumor Volume Segmentation and Pathologic Volume of Primary Human Solid Tumors | AJR - OBJECTIVE. The purpose of this study was to establish the correlation and reliability among the path...
8. MedSAM 2 Explained: Segment Anything in Medical Imaging - Discover how MedSAM2 brings prompt-based segmentation to CT, MRI, PET, and live ultrasound, cutting ...
9. SAM 2: Segment Anything Model 2 - Discover SAM 2, the next generation of Meta's Segment Anything Model, supporting real-time promptabl...
10. sam2 · PyPI - SAM 2: Segment Anything in Images and Videos
11. Visual Foundation Models for Medical Image Analysis - The analysis of 3D medical images is crucial for advancing clinical responses, disease tracking, and...
12. Model Overview - MONAI offers serveral frameworks, and we are adding to them all the time. Here, you’ll find the info...
13. MONAI Core - MONAI Core is the flagship library of Project MONAI, providing powerful capabilities for medical AI ...

14. MedSAM2: Segment Anything in 3D Medical Images and Videos - MedSAM2 introduces a robust foundation model for promptable segmentation in 3D medical images and te...
15. MedSAM2: Segment Anything in 3D Medical Images and Videos - MedSAM2 is a promptable segmentation network with an image encoder, a prompt encoder, a memory atten...
16. Lifelong Multi-Agent Path Finding for Online Pickup and ... - by H Ma · 2017 · Cited by 460 — In this section, we present first a simple decoupled MAPD algorithm,...
17. Multi-Agent Path Finding for Online Warehouses - by A Harmanec · Cited by 1 — presented two decoupled MAPD algorithms, To- ken Passing (TP) and the i...
18. [PDF] Lifelong Multi-Agent Path Finding for Online Pickup and ...
19. [PDF] A Multi-Label A* Algorithm for Multi-Agent Pathfinding - CMU
20. Multi-Agent Path Finding and Multi-Agent Pickup and Delivery - by G Lodigiani · Cited by 6 — Token Passing (TP) is a decoupled algorithm based on a token, a synchr...
21. Multi-agent Path Planning Based on Conflict- ... - by Y Bai · 2025 · Cited by 11 — Conflict-based search (CBS) stands as an optimal and com- plete two-...
22. Comparative Analysis of Conflict-Based Search Heuristics ... - Conflict-Based Search (CBS), a two-level algorithm that employs both a high-level and low-level sear...
23. Optimize inference using torch.compile() - Depending on the model and the GPU, torch.compile() yields up to 30% speed-up during inference. To u...
24. Supercharge Your PyTorch Image Models: Bag of Tricks to 8x ... - By the end of the post you'll learn how to supercharge the inference speed of any image models from ...
25. Faster PyTorch Model Inference Using ONNX Runtime - Osi - In this blog post, you will learn how to convert a Pytorch state-dictionary model into ONNX format f...
26. AI can understand your medical records: A new language ... - Researchers believe that Norway has now made a significant step forward in the use of artificial int...
27. Domain-Specific Pretraining of NorDeClin-Bidirectional ... - JMIR AI - Background: Accurately assigning ICD-10 codes is critical for clinical documentation and epidemiolog...

28. Vectors/norlm/norbert - Nordic Language Processing Laboratory
29. Training and Evaluating Norwegian Sentence Embedding ... - by BIU Nødland · 2023 — We demonstrate a new way to compare the various existing Norwegian language ...